

Software-Engineering Discipline Systems for Coding Agents

GOAL: Curate a set of software engineering frameworks (skills and commands) to help guide the AI Fellows' (and their coding assistants) in producing agentic systems that can be promoted to PROD as robust, secure and maintainable code.

Note on popularity figures: GitHub star counts here are noisy and frequently inflated — roundups report the same repo anywhere from ~40k to ~256k stars depending on when and how they counted. [Taskade verified counts against the GitHub API on July 1, 2026](#); others repeat higher unverified figures. Treat the ranking as ordinal, not cardinal.

Contents

1. Developers and Their Proposed Software Engineering for AI Frameworks (ordered by popularity)
 - Practitioner Methodologies & Course-Shipped Systems
 - Institutional / Non-Single-Person Systems
2. Published Comparisons
 - A Structural Distinction Worth Keeping in Mind
 - Framework Repository Reference
3. Implementing the Preferred Package: Superpowers + Compound Engineering + OpenSpec + karpathy-skills
4. Verifying a Successful Install (with figures)
5. How the User Typically Invokes Each Tool

1. Developers and Their Proposed Software Engineering for AI Frameworks (ordered by popularity)

1. Jesse Vincent (obra; Prime Radiant) — Superpowers

URL: github.com/obra/superpowers

Stars: ~40k → 256k (varies by source)

Idea: Enforced TDD (red/green/refactor), spec-first brainstorming, subagent-driven development, and “anti-rationalization” gates. Composable skills library on Anthropic’s official marketplace; runs on Claude Code, Codex, Cursor, OpenCode, Copilot, Gemini.

2. Matt Pocock (Total TypeScript) — mattpocock/skills

URL: github.com/mattpocock/skills · aihero.dev/skills

Stars: ~87k → 176k

Idea: 40+ composable skills for real workflows — TDD, diagnosis, triage, architecture review. Installs via `npx skills add mattpocock/skills`; wires to your issue tracker.

3. Garry Tan (Y Combinator CEO) — gstack

URL: github.com/garrytan/gstack

Stars: ~66k → 118k

Idea: Turns Claude Code into a “virtual engineering team” — 23 specialist roles + 8 tools as slash commands, on a Think → Plan → Build → Review → Test → Ship → Reflect loop. OWASP/STRIDE security review. MIT.

4. Addy Osmani (Google Chrome eng director) — agent-skills

URL: github.com/addyosmani/agent-skills

Stars: ~68k → 75k

Idea: 20+ skills + 7–8 slash commands mapped to the lifecycle (`/spec`, `/plan`, `/build`, `/test`, `/review`, `/ship`). Encodes Google engineering culture: Hyrum’s Law, Beyoncé Rule, test pyramid (80/15/5), Shift Left.

5. Forrest Chang (encoding Andrej Karpathy) — karpathy-skills (by raw stars, the #1 skills repo)

URL: github.com/multica-ai/andrej-karpathy-skills

Stars: ~188k–194k on the multica-ai mirror as of July 2026 (Star History: 194.6k, global rank #27). Two-account history: original forrestchang (~91k) + multica-ai mirror (~132k in May) were often summed to ‘220,000+ combined’. The ‘156,000’ figure in the [Govelocity roundup](#) was a June snapshot — already stale.

Idea: A single ~65–70-line CLAUDE.md/SKILL.md, zero dependencies, works with any agent reading a CLAUDE.md/SKILL.md. Four rules: Think Before Coding (state assumptions, ask when uncertain, surface tradeoffs), Simplicity First (no abstractions/error-handling beyond what was asked), Surgical Changes (touch only what you must; no orthogonal edits), Goal-Driven Execution (loop until verified). Targets the exact failures Karpathy named: silent assumptions, over-engineering, unrequested changes. Biases toward caution over speed — possibly over-calibrated for simple edits.

Caveats: Karpathy did not write it and has not endorsed it; and the repeated ‘65% → 94% accuracy’ claim is a [misattribution](#) from separate Anthropic experiments.

On ranking: #1 by raw stars, but listed 5th because this section orders by durable multi-signal adoption (marketplace presence, cross-tool integration, sustained usage), and this repo’s counts are especially noisy. Read it as ‘#1 by stars,’ not ‘#1 by depth of discipline’ — it is one file of guardrails, not a full methodology like Superpowers or Compound Engineering.

6. Harper Reed — “My LLM Codegen Workflow” (blog method)

URL: harper.blog/2025/02/16/my-llm-codegen-workflow-atm

Idea: Spec → Plan → Execute as discrete loops. Idea-honing produces spec.md; a reasoning model turns it into prompt_plan.md + todo.md; a code-gen agent implements each step test-first. Amplified by [Simon Willison](#).

7. Geoffrey Huntley — “Ralph Wiggum” / the Ralph loop

URL: ghuntley.com/ralph · ghuntley.com/loop

Idea: Treat everything as a self-improving agent loop driven by an AGENT.md. The agent updates its own instructions as it learns build/run commands; guardrails stop it assuming code is unimplemented before searching.

8. Brian Madison — BMAD-METHOD

URL: github.com/bmad-code-org/bmad-method

Stars: ~49.5k

Idea: “Breakthrough Method for Agile AI-Driven Development.” 19+ role agents (PM, Architect, Scrum Master, Dev, QA) and 50+ named workflows across the full SDLC. Native Claude Code integration; 42+ platforms. MIT.

Practitioner Methodologies & Course-Shipped Systems

Named, opinionated systems built by individual practitioners — shipping as a plugin, a course with bundled skills/commands, or a memory tool — that encode a specific engineering discipline rather than a generic skill catalog. Added at the reader’s suggestion.

Kieran Klaassen (GM of Cora at Every) — Compound Engineering plugin

URL: github.com/EveryInc/compound-engineering-plugin

Stars: ~6.8k → 10k+ (Every reports 10,000+; used by engineers at Google and Amazon)

Idea: “Each unit of engineering work should make subsequent units easier — not harder.” 80% planning/review, 20% execution. Ships as a Claude Code plugin with a slash command per loop step: /ce-plan, /ce-work, /ce-code-review (multi-agent review of security, architecture, quality), /ce-compound (codify learnings). The loop expanded from four steps to eight: Ideate → Brainstorm → Plan → Work → Review → Polish → Compound.

IndyDevDan — Tactical Agentic Coding (TAC) & Principled AI Coding

URL: agenticengineer.com/tactical-agentic-coding · github.com/disler

Format: A paid course (not a repo) shipping skills and commands, plus an extended “Agentic Horizon” lesson set.

Idea: “Build the system that builds the system” — move from ‘in the loop’ prompting to ‘out of the loop’ autonomous systems, encoding your practices so agents ship how you would. Frameworks: PITER, Closed Loop Prompts, separate Review/Documentation agents, the Core Four (Context, Model, Prompt, Tools), and ZTE (‘zero touch engineering’).

Tracks four agentic-coding KPIs.

Steve Yegge — Beads (bd) coding-agent memory system

URL: github.com/steveyegge/beads · [announcement](#)

Idea: A dependency-graph issue tracker for agents. Point your AGENTS.md/CLAUDE.md at the bd tool with one line and agents switch from brittle markdown plans to an issue graph — granting continuity across sessions and long-horizon planning so no discovered work is lost. From the co-author of the ‘Vibe Coding’ book.

Gota Kirin (gotalab) — cc-sdd

URL: github.com/gotalab/cc-sdd

Idea: One command installs an agentic SDLC workflow as Agent Skills — discovery, requirements, design, tasks, and autonomous implementation with per-task independent review. Same 17-skill set across 8 AI coding agents.

AWS — Kiro (spec-driven IDE + portable skills)

URL: kiro.dev/docs/cli/skills

Idea: An AWS-built agentic IDE organized around a .kiro/specs/ directory (requirements → design → tasks read as first-class input each session), plus portable skill packages for PR review, infra deployment, and coding standards.

Related philosophy (Every): Kieran’s colleague [Dan Shipper](#) (Every CEO) frames the surrounding management practice — ‘a four-step engineering process for software teams that don’t write code’ — building on the same compound-engineering loop.

Institutional / Non-Single-Person Systems

- GitHub Spec Kit — spec-driven development (/speckit.constitution → specify → clarify → plan → tasks → implement), ~115k–122k stars, agent-agnostic across 30+ agents. [github.github.com/speckit](https://github.com/speckit)
- AGENTS.md — OpenAI-originated, now vendor-neutral open standard (60,000+ repos); read natively by Codex, Cursor, Copilot, Gemini CLI, Aider, Windsurf, Zed. [Codersera](#)
- OpenSpec (~55.9k), GSD / get-shit-done (~64k), Task Master AI (~27.7k). [SSOJet](#)
- Anthropic official skills repo (Skill Creator + document skills) — baseline reference. github.com/anthropics/skills

2. Published Comparisons

Comparison	What it covers
Beyond Vibe Coding: 7 Production Agent Skills Frameworks	The most explicit head-to-head — mattpocock/skills , Superpowers , Spec Kit , addyo/mani/agent-skills , skills.sh , Cloudflare Skills , Vercel Labs Skills , with ratings and a stars/skills/install/learning-curve table.
Superpowers vs. BMAD vs. SpecKit vs. OpenSpec vs. GSD	Treats the five “frameworks (not tools)” and argues their approaches differ enough that picking wrong wastes weeks.
7 Spec-Driven Development Tools	Spec Kit , Kiro , OpenSpec , BMAD , Task Master — pricing/license/star/best-for table across the spec-driven camp.
OpenSpec vs GitHub Spec Kit	A focused two-way spec-driven comparison.
AGENTS.md vs CLAUDE.md vs Cursor Rules: 2026 Guide	Compares the instruction-file conventions — the markdown layer beneath the skill systems.
Best Agentic Coding Method in 2026? (video)	Superpowers vs BMAD vs OpenSpec video comparison.

Curated indexes that also function as comparison surfaces: [hesreallyhim/awesome-claude-code](#), [Govelomatrix's](#) 20 most-starred repos, [Taskade's](#) API-verified ranking, [Firecrawl's](#) tested list.

A Structural Distinction Worth Keeping in Mind

The comparisons converge on a taxonomy of four overlapping camps:

- Spec-driven — Spec Kit, OpenSpec, Kiro, cc-sdd, Harper Reed's method. Front-load a rigorous spec, then execute against it.
- Methodology / skill-library — Superpowers, gstack, agent-skills, mattpocock/skills, IndyDevDan's TAC. Encode senior-engineer discipline as auto-loading skills plus verification gates.
- Multi-agent-role — BMAD, gstack's team-of-specialists. Assign distinct role agents (PM, architect, QA, security) across a workflow.
- Compounding / self-improving loop — Kieran Klaassen's Compound Engineering, Geoffrey Huntley's Ralph loop, Steve Yegge's Beads. The system learns from each cycle (codified learnings, a self-editing AGENT.md, or a persistent issue graph) so the next unit of work is easier and past mistakes aren't repeated.

These camps overlap; the strongest systems combine them (Compound Engineering is spec-driven, multi-agent-role, and a compounding loop at once). Most sources agree Superpowers is the most-adopted single-person skill library, Compound Engineering the most-cited compounding-loop methodology, and AGENTS.md the lowest-common-denominator instruction standard everything else layers on top of — Codersera, Dailiyaiworld.

Framework Repository Reference

Framework	Author / Owner	Repository / Primary URL
Superpowers	Jesse Vincent (obra)	github.com/obra/superpowers
BMAD-METHOD	Brian Madison	github.com/bmad-code-org/bmad-method
GitHub Spec Kit	GitHub	github.com/github/spec-kit
OpenSpec	Fission AI / community	github.com/Fission-AI/OpenSpec
Kiro	AWS	github.com/kirodotdev/Kiro
Compound Engineering plugin	Kieran Klaassen (Every)	github.com/EveryInc/compound-engineering-plugin
gstack	Garry Tan	github.com/garrytan/gstack
Tactical Agentic Coding (TAC)	IndyDevDan	agenticengineer.com/tactical-agentic-coding (course)
Beads (bd) issue graph	Steve Yegge	github.com/steveyegge/beads
mattpocock/skills	Matt Pocock	github.com/mattpocock/skills
agent-skills	Addy Osmani	github.com/addyosmani/agent-skills
karpthy-skills (~188k–194k ★, #1 by raw stars)	Forrest Chang	github.com/multica-ai/andrej-karpthy-skills
cc-sdd	Gota Kirin (gotalab)	github.com/gotalab/cc-sdd
Anthropic official skills	Anthropic	github.com/anthropics/skills

Note: Kiro is primarily distributed as a downloadable IDE at kiro.dev; the repository above hosts its public issue tracker and docs. IndyDevDan's TAC is a paid course rather than an open repository — his related open-source prompt work lives at github.com/disler.

3. Implementing the Preferred Package: Superpowers + Compound Engineering + OpenSpec + karpathy-skills

A complete, step-by-step guide to standing up a new VS Code project with all four systems available together. They are complementary layers, not competitors:

- OpenSpec — the spec layer. Front-loads a rigorous, reviewable spec (the what/why) before code is written.
- Compound Engineering — the workflow-loop layer. Plan → Work → Review → Compound, where each cycle documents learnings so future work gets easier.
- Superpowers — the engineering-discipline layer. Auto-triggering TDD and subagent-driven development skills (the how).
- karpathy-skills — the guardrail layer. Always-on behavioral rules (the don't — no silent assumptions, no over-engineering, no orthogonal edits).

A note on overlap: Compound Engineering and OpenSpec both include a planning step, and Compound's /ce-code-review overlaps with Superpowers' test gates. That's fine — use OpenSpec for heavyweight committed specs and Compound's lighter loop for daily iteration, or lead with Compound and reach for OpenSpec only on larger changes. Step 7 shows one coherent combination; adjust to taste.

How these tools actually load (read this first)

None of these are VS Code Marketplace extensions. They all plug into Claude Code, which runs inside VS Code via the Claude Code extension. Three of the four are Claude Code plugins (Superpowers, Compound Engineering, and karpathy via its recommended path) — installed once, available across all projects. OpenSpec is an npm CLI that must be initialized per project, writing spec files and /openspec-* commands into that repo. **Skills** load from ~/.claude/skills/ (personal) or <repo-root>/claude/skills/ (project-scoped, committed); skills **trigger automatically** while /ce-* and /openspec-* commands are typed into the chat. Bottom line: no repositories need to be git-cloned — three install via the plugin marketplace, OpenSpec via npm.

Prerequisites

- VS Code installed.
- Node.js ≥ 20.19.0 — required by OpenSpec ([README](#)). Check: node --version.
- Claude Code installed and authenticated, with its VS Code extension enabled. Claude Code reads the plugins, skills, and CLAUDE.md.
- git for version-controlling the project and its committed skills.
- (Optional) Bun — only to convert the Compound Engineering plugin to OpenCode/Codex formats; not needed for Claude Code.

Step 1 — Create the VS Code project

```
mkdir my-agentic-project
cd my-agentic-project
git init
code . # opens the folder in VS Code
```

Open the integrated terminal (Ctrl+` / Cmd+`) and start Claude Code with c laude. Commands beginning with / are typed into the Claude Code chat; npm/curl/openspec commands go in the shell.

Step 2 — Install Superpowers (Claude Code plugin, global)

From within Claude Code ([README](#)):

```
/plugin install superpowers@claude-plugins-official
```

If it isn't on the official marketplace in your version, register the author's marketplace first:

```
/plugin marketplace add obra/superpowers-marketplace
```

```
/plugin install superpowers@superpowers-marketplace
```

Installs globally; skills then trigger automatically in every project. (Cursor: `/add-plugin superpowers.`)

Step 3 — Install Compound Engineering (Claude Code plugin, global)

From within Claude Code ([README](#)):

```
/plugin marketplace add https://github.com/EveryInc/compound-engineering-plugin
```

```
/plugin install compound-engineering
```

Adds the core workflow slash commands (all prefixed `/ce-`):

- `/ce-plan` — turn a feature idea into a detailed implementation plan
- `/ce-work` — execute the plan with git worktrees and task tracking
- `/ce-code-review` (and `/ce-doc-review`) — multi-agent review (security, architecture, quality) before merging
- `/ce-compound` — document learnings so future work is easier
- Additional `/ce-` commands cover the wider loop: `/ce-ideate`, `/ce-brainstorm`, `/ce-debug`, `/ce-worktree`, `/ce-setup`, `/ce-update`, `/ce-simplify-code`, `/ce-commit`, `/ce-commit-push-pr`, `/ce-proof`, `/ce-polish`, `/ce-promote` (plus `/lfg`).

After installing plugins, run `/reload-plugins` (or restart the IDE / Claude Code session) so the new commands register — they load at session start.

(Optional, non-Claude agents) Convert with the Bun CLI: `bunx @every-env/compound-plugin install compound-engineering --to opencode` (writes to `~/opencode`) or `--to codex` (`~/codex/prompts` and `~/codex/skills`). Both experimental.

Step 4 — Install karpathy-skills (behavioral guardrails). Pick ONE option.

Option A — Plugin (recommended, global) — from within Claude Code ([README](#)):

```
/plugin marketplace add forrestchang/andrej-karpathy-skills
```

```
/plugin install andrej-karpathy-skills@karpathy-skills
```

Option B — Per-project CLAUDE.md (committed to this repo) — in the shell at the project root:

```
# new project (creates the file):
curl -o CLAUDE.md
https://raw.githubusercontent.com/forrestchang/andrej-karpathy-skills/main/CLAUDE.md
# OR append to an existing CLAUDE.md:
echo "" >> CLAUDE.md
curl https://raw.githubusercontent.com/forrestchang/andrej-karpathy-skills/main/CLAUDE.md >>
CLAUDE.md
```

Option B is more transparent for a shared repo (rules live in a version-controlled CLAUDE.md); Option A auto-applies everywhere.

Step 5 — Install and initialize OpenSpec (spec layer, per-project)

Install the CLI globally once, in the shell ([README](#)):

```
npm install -g @fission-ai/openspec@latest
```

Then initialize it inside the project directory:

```
cd my-agentic-project
openspec init
```

This registers OpenSpec's slash commands and creates an openspec/ folder. Changes live under openspec/changes/<name>/ (proposal.md, specs/, design.md, tasks.md); completed work archives to openspec/changes/archive/. Refresh agent instructions anytime with openspec update. Restart Claude Code (or /reload-plugins) after init so the commands register.

- /openspec-propose <what-to-build> — draft a spec/proposal
- /openspec-apply-change — implement the tasks in the spec
- /openspec-archive-change — archive the completed change
- Also available: /openspec-explore, /openspec-sync-specs, /openspec-update-change.

A note on command names: OpenSpec's docs use a canonical /opsx: shorthand, but when installed in Claude Code the commands register under the /openspec-* names shown above. Exact spelling can vary by tool — type /openspec and let autocomplete confirm what is registered.

Step 6 — Verify everything is live

```
ls ~/.claude/skills/ # personal plugins/skills
ls .claude/skills/ 2>/dev/null # project-scoped, if committed
openspec --version
cat CLAUDE.md | head # if you chose karpathy Option B
```

In the Claude Code chat, run /plugin to confirm superpowers, compound-engineering, and andrej-karpathy-skills all appear; type /ce- and /openspec- to confirm both command families autocomplete.

Step 7 — Recommended end-to-end workflow

- Spec (OpenSpec): /openspec-propose add-user-auth, then review/edit proposal.md, specs/, design.md, tasks.md. Use for substantial changes; for quick iterations skip to the Compound loop.
- Plan (Compound): /ce-plan turns the spec/idea into a concrete implementation plan with worktrees and task tracking.
- Work (Compound + Superpowers + karpathy): /ce-work (or /openspec-apply-change). Superpowers auto-triggers TDD and subagent development; karpathy's rules block silent assumptions, over-engineering, and unrelated edits — you don't invoke these two.
- Review (Compound): /ce-code-review for multi-agent security/architecture/quality review; corroborate with Superpowers' test gates. Confirm the diff is surgical.
- Compound (Compound): /ce-compound documents what was learned so the next cycle is faster.
- Archive (OpenSpec): /openspec-archive-change moves the finished change into a dated openspec/changes/archive/ folder for durable history.

Suggested repo layout after setup

```
my-agentic-project/
.claude/skills/ # (optional) project-scoped skills committed for the team
CLAUDE.md # karpathy four rules (Option B) + project notes
openspec/
changes/add-user-auth/ -> proposal.md, specs/, design.md, tasks.md
```

```
changes/archive/2026-07-24-add-user-auth/  
src/ # your actual code
```

(Superpowers, Compound Engineering, and karpathy Option A live under ~/.claude/ per-machine, so they don't appear in the repo tree.)

Team / reproducibility notes

- Commit CLAUDE.md, the openspec/ directory, and any .claude/skills/ you want shared. Teammates get spec history and guardrails on clone; they run openspec init/update locally and install the three plugins once each.
- Global vs. project: Superpowers, Compound Engineering, and (Option A) karpathy live in ~/.claude/ (per-machine); OpenSpec's artifacts are per-repo and travel with the code.
- Updating: re-run the plugin install commands for the three plugins; run npm install -g @fission-ai/openspec@latest then openspec update for OpenSpec.

Sources: [Superpowers README](#) · [Compound Engineering README](#) · [OpenSpec README](#) · [karpathy-skills README](#) · [Where Claude skills are stored](#).

4. Verifying a Successful Install (with figures)

Once you have installed the four-tool package, use the screenshots below as a reference for what a correct, fully loaded install looks like. They are from a real Windows install (Git Bash / MINGW64, VS Code integrated terminal). If your environment matches these, the package is installed and loaded correctly.

Figure 1 — Plugins panel: all three plugins enabled

Run `/plugins` in Claude Code. A correct install shows `andrej-karpathy-skills`, `superpowers`, and `compound-engineering` each marked “✓ enabled.” (MCP entries like `mermaid/pencil` are unrelated.)

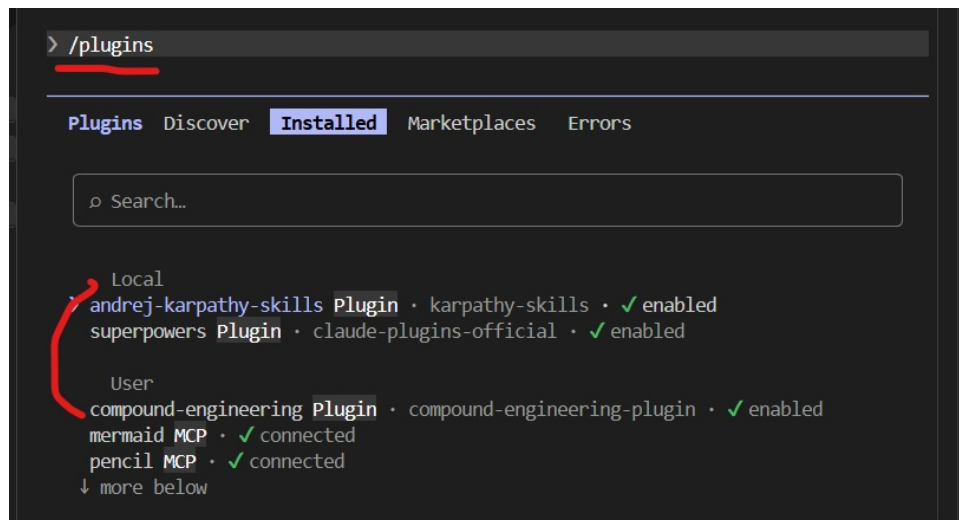


Figure 1. The `/plugins` panel with all three package plugins enabled.

Figure 2 — Plugin install log

Each plugin install should confirm success. Note the two “Run `/reload-plugins` to apply” messages — that is the reload step from Section 3. (“`compound-engineering ... is already installed`” is harmless — the marketplace add already pulled it in.)

Figure 3 — openspec init output

In the shell (not the Claude chat), `npm install -g @fission-ai/openspec@latest` then `openspec init` should report "6 skills and 6 commands in .claude/" and "Setup complete for Claude Code." Notice "Restart your IDE for the new commands to take effect." The `ls ~/.claude/skills/` confirms the six `openspec-*` skill files were created.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ... bash + v [ ] [ ] ...
biafo@--4060-- MINGW64 /a/Admin/Jobs/2026/Jay_Project/SE_for_AI/Test_Bed (master)
$ npm install -g @fission-ai/openspec@latest
  run `npm fund` for details

biafo@--4060-- MINGW64 /a/Admin/Jobs/2026/Jay_Project/SE_for_AI/Test_Bed (master)
$ openspec init
Note: OpenSpec collects anonymous usage stats. Opt out: OPENSPEC_TELEMETRY=0

Welcome to OpenSpec
Detected tool directories: Claude Code (pre-selected for first-time setup)
✓ Select tools to set up (34 available) Claude Code
| OpenSpec structure created
✓ Setup complete for Claude Code

OpenSpec Setup Complete

Created: Claude Code
6 skills and 6 commands in .claude/
Config: openspec/config.yaml (schema: spec-driven)

Getting started:
  Start your first change: /opsx:propose "your idea"

Learn more: https://github.com/Fission-AI/OpenSpec
Feedback:   https://github.com/Fission-AI/OpenSpec/issues

Restart your IDE for the new commands to take effect.

biafo@--4060-- MINGW64 /a/Admin/Jobs/2026/Jay_Project/SE_for_AI/Test_Bed (master)
$ ls ~/.claude/skills/
ls: cannot access '/c/Users/biafo/.claude/skills/': No such file or directory

biafo@--4060-- MINGW64 /a/Admin/Jobs/2026/Jay_Project/SE_for_AI/Test_Bed (master)
$ ls ~/.claude/skills/
openspec-apply-change/  openspec-explore/  openspec-sync-specs/
openspec-archive-change/ openspec-propose/  openspec-update-change/

biafo@--4060-- MINGW64 /a/Admin/Jobs/2026/Jay_Project/SE_for_AI/Test_Bed (master)
$ openspec --version
```

Figure 3. openspec init reporting success, the restart notice, and the six openspec-* skills.

Figure 4 — Compound Engineering autocomplete (/ce-)

Type `/ce` in the Claude chat. Autocomplete lists the real commands — `/ce-plan`, `/ce-work`, `/ce-proof`, `/ce-debug`, etc., each tagged (compound-engineering). Direct proof the prefix is `/ce-`.

```
biafo@--4060-- MINGW64 /a/Admin/Jobs/2026/Jay_Project/SE_for_AI/Test_Bed (master)
$ claude_a
```

Welcome back Louis!



Opus 4.8 (1M context) · Claude Max ·
biaforem@gmail.com's Organization
A:\Admin\Jobs\2026\Jay_Project\SE_for_AI\Test_Bed

Tips for getting started
Run `/init` to create a CLA...

What's new
Added `--safe-mode` flag ...
Added `/cd` command to mo...
Added a `disableBundledSk...`
[/release-notes for more](#)

Opus 4.8 is here! Now defaults to high effort · `/effort xhigh` for your hardest tasks

```
> /ce
```

<code>/ce-plan</code>	(compound-engineering) Create structured plans for multi-step tasks -- software features, re...
<code>/ce-work</code>	(compound-engineering) Execute work efficiently while maintaining quality and fin...
<code>/ce-proof</code>	(compound-engineering) Run human-in-the-loop review loops over markdown via Proof (proofed...
<code>/ce-debug</code>	(compound-engineering) Systematically find root causes and fix bugs. Use when debugging ...
<code>/ce-setup</code>	(compound-engineering) Diagnose and configure compound-engineering environment. Checks CLI ...
<code>/ce-update</code>	(compound-engineering) Check if the compound-engineering plugin is up to date and...
<code>/ce-polish</code>	(compound-engineering) Start the dev server, open the feature in a browser, and iterate on...
<code>/ce-commit</code>	(compound-engineering) Create a git commit with a clear, value-communicating message. Us...
<code>/ce-ideate</code>	(compound-engineering) Generate and critically evaluate grounded ideas about a topic. Use wh...
<code>/ce-promote</code>	(compound-engineering) Draft user-facing announcement and marketing copy for a feature...

Figure 4. Autocomplete showing `/ce-plan`, `/ce-work`, `/ce-proof` and other compound-engineering commands.

Figure 5 – OpenSpec autocomplete (/openspec-)

Type `/open` in the Claude chat. Autocomplete lists `/openspec-explore`, `/openspec-propose`, `/openspec-sync-specs`, `/openspec-apply-change`, `/openspec-update-change`, and `/openspec-archive-change` — the command family Claude Code registers.



Figure 5. Autocomplete showing the `/openspec-*` command family registered in Claude Code.

Post-install checklist

- Plugins enabled — `/plugins` shows superpowers, compound-engineering, and andrej-karpathy-skills all enabled (Fig 1).
- Installs confirmed — each plugin install reported success (Fig 2).
- OpenSpec initialized — `openspec init` reported "6 skills and 6 commands in .claude/" and `openspec --version` returns a version (Fig 3).
- Reloaded / restarted — you ran `/reload-plugins`, or restarted claude, after installing (fixes "Unknown command").
- Compound commands resolve — typing `/ce` autocompletes `/ce-plan`, `/ce-work`, ... (Fig 4).
- OpenSpec commands resolve — typing `/openspec` autocompletes `/openspec-propose`, ... (Fig 5).
- Superpowers / karpathy need no command — start a task; Claude should ask clarifying questions before coding and keep diffs surgical.

If every item checks out, the package is correctly installed and loaded.

Figures 1–5 are from a verified Windows install.

5. How the User Typically Invokes Each Tool

A recurring source of confusion is that these four tools are triggered in fundamentally different ways. Two fire automatically based on what you're doing; two require an explicit command. Knowing which is which is the difference between the package working invisibly and you wondering why nothing happened.

The setting: in this project Claude Code most often runs from the CLI in a VS Code integrated terminal — you open the terminal (Ctrl+` / Cmd+`) and start it with `c laude`. VS Code is the editor host; Claude is a terminal process. (Some people use the Claude Code side-panel/extension instead, but the terminal is the common case.) Once Claude is running there are two kinds of input: slash commands typed into the chat (read by Claude, not the shell) and natural-language requests typed into the chat (what cause auto-triggered skills to fire). Shell commands (`npm`, `git`, `openspec init`, test runners) are separate — typed at the terminal prompt.

The ideal: the user should rarely type any command at all

Everything below documents what each command does and how it fires — but in daily use the goal is that the user names an outcome in plain language and Claude drives the commands itself. This works when a discipline file (this project's `SE_Discipline.md`) is loaded early in the session — imported from `CLAUDE.md`, or installed as an always-on skill — so Claude reads it as its operating playbook and knows which `/ce-*` and `/openspec-*` command to run at which moment.

- The user says: "Add OAuth login."
- Claude, guided by `SE_Discipline.md`, self-invokes the pipeline: triages the task, runs `/openspec-propose` and pauses for spec approval, then `/ce-plan`, sets up a worktree, implements test-first (Superpowers + karpathy engage automatically), runs `/ce-code-review`, then `/ce-compound` and `/openspec-archive-change`.
- The user's only jobs are stating the goal and hitting the human checkpoints the discipline requires (approve the spec, the design, the merge).

So the slash commands are best understood as Claude's vocabulary, not the user's homework. The user may type them to force a specific step, but a well-elaborated `SE_Discipline.md` means they usually won't have to.

Prerequisite: `SE_Discipline.md` must actually be in Claude's context each session — @import it from the repo's `CLAUDE.md`, paste a one-line "Read `SE_Discipline.md` before starting" at session open, or package it as an auto-loading project skill. If it is not loaded, you fall back to the manual invocation patterns below.

Invocation at a glance

Tool	Trigger type	How it gets invoked (ideal → fallback)
Superpowers	Automatic (context-detected)	Fires on its own from a plain-language goal — no command, ever.
karpathy-skills	Always-on (ambient)	Nothing — the rules apply to every Claude response once installed.
Compound Engineering	Explicit (slash command)	Ideal: Claude runs <code>/ce-*</code> for you per <code>SE_Discipline.md</code> . Fallback: you type them.
OpenSpec	Explicit (slash + CLI)	<code>/openspec-*</code> in chat for the workflow; <code>openspec</code> in the shell for setup/maintenance.

1. Superpowers — invoked by describing the work (automatic)

You do not type a command. Its skills are “mandatory workflows, not suggestions”; the agent “checks for relevant skills before any task” and they “trigger automatically” ([Superpowers](#)).

- To start a build: state what you’re making in plain language. Superpowers steps back and asks what you’re really trying to do (brainstorming) instead of jumping to code.
- To implement: approve the design; writing-plans → subagent-driven-development → test-driven-development engage in turn.
- To review: finish a task — requesting-code-review fires between tasks.
- To debug: report a bug — systematic-debugging engages.

The user’s real “invocation” of Superpowers is good problem statements. You can also ask for a skill by name to force it.

2. karpathy-skills — invoked by existing (always-on)

There is no invocation step. Once installed (plugin, or a CLAUDE.md committed to the repo), the four rules — Think Before Coding, Simplicity First, Surgical Changes, Goal-Driven Execution — apply to every response ([karpathy-skills](#)).

- To benefit: nothing — just work normally. They surface as clarifying questions before coding and surgical diffs.
- To adjust: with the CLAUDE.md route, edit that file; the new rules apply on Claude’s next response and travel with the repo for teammates.

3. Compound Engineering — invoked by a /ce- command (explicit)

Nothing happens until a slash command is issued — ideally by Claude on your behalf, otherwise by you ([Compound Engineering](#)):

- /ce-plan — turn an idea into a detailed implementation plan
- /ce-work — run the plan with worktrees and task tracking
- /ce-code-review — multi-agent pre-merge review
- /ce-compound — document learnings so the next cycle is easier

4. OpenSpec — invoked in two places: /openspec-* in chat and openspec in the shell (explicit)

OpenSpec has both a chat surface and a terminal surface ([OpenSpec](#)):

- Setup/maintenance (terminal): openspec init, openspec update, openspec --version.
- Everyday workflow (/openspec-* in chat): /openspec-propose <slug>, /openspec-apply-change, /openspec-archive-change; also /openspec-explore, /openspec-sync-specs, /openspec-update-change.

Putting it together — a single feature, start to finish

Ideal mode (SE_Discipline.md loaded — Claude self-drives). The user speaks once; Claude runs the commands and pauses only at human checkpoints:

- User: “Add OAuth login.” That single sentence is the only required input.
- Claude triages it as substantial and runs /openspec-propose add-oauth-login itself, then pauses for spec approval.

- On approval, Claude runs `/ce-plan`, sets up a worktree, implements test-first (Superpowers TDD + karpthy surgical diffs engage automatically).
- Claude runs `/ce-code-review`, reports findings, pauses for merge approval.
- On approval, Claude runs `/ce-compound` and `/openspec-archive-change`.

The user typed zero slash commands — they stated a goal and answered three approval prompts. That is the target experience.

Manual mode (fallback). If the discipline file isn't loaded (or you want to force a step), the user drives each command by hand: state goal → `/openspec-propose` → `/ce-plan` → `/ce-work` (or `/openspec-apply-change`) → `/ce-code-review` → `/ce-compound` → `/openspec-archive-change`.

The distinction to remember: Superpowers and karpthy always fire on their own. Compound and OpenSpec must be invoked by command — the only question is whether Claude invokes them for you (ideal) or you invoke them by hand (fallback).

Sources: [Superpowers README](#) · [Compound Engineering README](#) · [OpenSpec README](#) · [karpthy-skills CLAUDE.md](#).